

Notes on Tuning the Hyperopt Objective Function in Practice

Luis Mendez

OVERVIEW

The companion note (*Other SMBO Models*) covers what `rand.suggest`, `anneal.suggest`, and `tpe.suggest` do algorithmically. This note is about a narrower, more practical problem that shows up once those algorithms are pointed at a real objective function: `02-Search-algorithms.ipynb` tunes an `XGBClassifier` on the breast cancer dataset with all three algorithms, and in every run the trained model reaches a train ROC-AUC of exactly 1.0, while a 50-trial TPE search took 3h22m to finish. Neither number is intrinsic to the search algorithm; both trace back to how the objective function was written. Fixing the objective, not the search algorithm, is what moved the needle.

DIAGNOSIS 1: CPU OVERSUBSCRIPTION

The objective function scored each candidate with

```
cross_val_score(gbm, X_train, y_train,
                 scoring='accuracy', cv=3, n_jobs=4)
```

which forks 4 worker processes, one per fold-batch. Each worker holds an `XGBClassifier` built with its *default* thread count, i.e. “use every core available.” Four processes each trying to claim every core means the OS is constantly context-switching between more threads than there are physical cores to run them on — classic oversubscription. This is consistent with the wall-clock numbers observed: random search (cheaper, shallower trees on average) finished in 52 minutes, annealing was still running after 46 minutes at trial 28/50, and TPE took 3h22m for the same 50 trials. The fix is to pin the model itself to a single thread and let `cross_val_score` own all the parallelism:

```
params_dict = {
    ...
    'n_jobs': 1,
    'tree_method': 'hist',
}
```

`tree_method='hist'` (histogram-binned split finding) is also substantially faster per tree than XGBoost’s exact-greedy default, independent of the threading fix. Together, these make each trial cheaper, which is what actually buys a bigger effective search — not raising `max_evals` against the original, oversubscribed objective.

DIAGNOSIS 2: AN UNREGULARIZED CORNER OF THE SEARCH SPACE

A train ROC-AUC of 1.0 on every single trial, across three different search algorithms and search-space regions, is a sign the model has enough capacity to perfectly separate every training point regardless of which hyperparameters are drawn. The original space,

$$\{n_{\text{est}}, \text{depth}, \eta, \text{booster}, \gamma, \text{subsample}, \text{colsample}_{*}, \lambda\}, \quad (1)$$

constrains *how much* data and how many features each tree sees, but not *how small a leaf is allowed to be* or *how large a raw split-weight is allowed to grow*. Two parameters close that gap:

- **min_child_weight** (searched over $[1, 20]$): the minimum sum of instance Hessian weight required in a leaf before XGBoost will keep splitting. Low values let the tree carve out leaves around single outlying training points; raising the floor forces leaves to aggregate enough evidence to be trustworthy.
- **reg_alpha** (searched over $[0, 5]$): the L_1 penalty on leaf weights, complementing the L_2 penalty already present as `reg_lambda`. L_1 additionally pushes small leaf weights to exactly zero, pruning marginal splits that L_2 alone would only shrink.

Neither guarantees train AUC drops below 1.0 — with $n_{\text{estimators}}$ up to 2500 the model can still memorize — but the search is now able to *find* more conservative regions of the space if those regions generalize better, which it could not do before because those regions did not exist in the space at all.

DIAGNOSIS 3: NOISY 3-FOLD SCORING

With `cv=3` and no fixed fold assignment, two evaluations of the same hyperparameter set can land on different folds and disagree purely from sampling noise, on a dataset with only 398 training rows. Since `anneal.suggest` and `tpe.suggest` both use the trial history to decide where to sample next, they are partly reacting to that noise rather than to genuine differences between hyperparameter sets. Two changes address this:

```
cv = StratifiedKFold(
    n_splits=5, shuffle=True,
    random_state=0)
```

fixed once, outside the objective, and reused for every trial. Five folds instead of three shrinks the variance of the mean CV score; fixing the split (rather than letting

`cross_val_score` re-derive a fold assignment implicitly) means every trial is scored on exactly the same train/validation partitions, so differences in the reported score are attributable to the hyperparameters and not to which rows happened to land in which fold.

A FREE VARIANCE-REDUCTION STEP: ENSEMBLING THE THREE SEARCHES

`rand.suggest`, `anneal.suggest`, and `tpe.suggest` were run independently and each produces its own locally-optimal configuration; they do not agree exactly, since each is a different (partly stochastic) path through the same space. Rather than picking one, averaging the three fitted models' predicted probabilities,

$$\hat{p}(\mathbf{x}) = \frac{1}{3} \left(\hat{p}_{\text{rand}}(\mathbf{x}) + \hat{p}_{\text{anneal}}(\mathbf{x}) + \hat{p}_{\text{tpe}}(\mathbf{x}) \right), \quad (2)$$

is a zero-cost variance-reduction step at prediction time: no extra trials, no extra tuning, just three models whose errors are not perfectly correlated (each converged to a different region of hyperparameter space) getting averaged together. This is the same intuition behind bagging, applied post-hoc to independently-tuned models instead of independently-resampled datasets.

PRACTICAL GUIDANCE

- Before increasing `max_evals` to search harder, check whether the objective function itself is leaving performance (in this case, wall-clock time) on the table — nested parallelism between `cross_val_score` and the model's own thread count is an easy one to miss.
- A train metric stuck at a perfect score across every trial is a search-space smell, not just a modeling one: it means the space never offered the search algorithm a sufficiently regularized configuration to try.
- Fix the cross-validation folds once, outside the objective, whenever a search algorithm uses trial history to decide where to sample next — otherwise part of what it is “learning” is fold-assignment noise.
- When multiple search algorithms are run over the same space anyway (e.g. to compare them, as here), averaging their resulting models is a free way to use all of that compute instead of discarding two of the three results.